

Proiectul 2. Break the Login

DOBRICEAN IOAN DORIAN

351

Capitolul 1. Prezentarea Aplicației inițiale

1.1 Prezentarea în ansamblu a aplicației

Pentru aplicația solicitată a fost realizată o versiune inițială, fără implementarea mecanismelor de securitate. După cum este ilustrat în Figura 1, structura proiectului conține trei directoare principale. Directorul **backend** include fișierul *db.js*, responsabil de comunicarea cu baza de date, *server.js*, care gestionează request-urile, și fișierul *.env*, unde sunt stocate credențialele pentru baza de date și configurațiile de port. Backend-ul este construit folosind framework-ul ExpressJS [1].

În directorul **database** se regăsesc datele gestionate prin PostgreSQL, precum și fișierul *init.sql*, utilizat pentru inițializarea și popularea bazei de date. În ceea ce privește **frontend-ul**, acesta constă într-o aplicație simplă realizată în ReactJS [2], care include un singur ecran ce permite autentificarea, înregistrarea, schimbarea parolei și afișarea unui mesaj de bun venit.

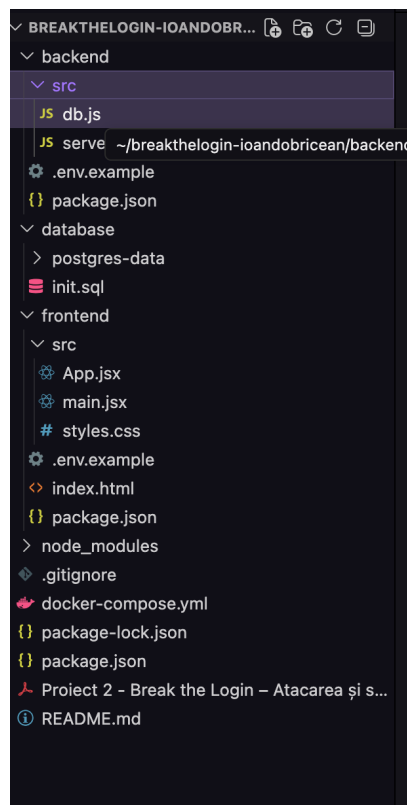


Figura 1. Structura Proiectului.

Pentru a porni aplicația, primul pas constă în inițializarea serviciilor necesare: PostgreSQL, backend-ul și frontend-ul, prin rularea comenzilor corespunzătoare:

```
>dobriceanioandorian-351 breakthelogs-in-andobricean cd backend
>dobriceanioandorian-351 backend docker compose up -d postgres

[+] Running 1/0
  ✓ Container authx-postgres Running
>dobriceanioandorian-351 backend
```

Figura 2. Deschidere PostgreSQL.

```
>dobriceanioandorian-351 backend npm run dev

> authx-backend@0.1.0 dev
> nodemon src/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/server.js`
AuthX backend vulnerabil pornit pe http://localhost:4000
```

Figura 3. Pornire backend.

```
VITE v5.4.21 ready in 99 ms

→ Local:   http://localhost:5173/
→ Network: http://172.20.10.8:5173/
→ press h + enter to show help
```

Figura 4. Pornire frontend.

Ulterior, aplicația de frontend se deschide, iar utilizatorul poate observa funcționalitățile disponibile: autentificare (login), înregistrare (register), recuperare parolă (forgot) și resetare parolă (reset).

The screenshot shows a web browser displaying the 'AuthX - autentificare' page. At the top, there are four tabs: 'login', 'register', 'forgot', and 'reset', with 'login' being the active tab. Below the tabs, there are two input fields: 'Email' with the placeholder 'ana@authx.local' and 'Parola' with the placeholder '123'. A 'Login' button is positioned below these fields. To the right of the login form, there is a section titled 'Panou AuthX' containing a list of links: 'Înregistrați un cont nou', 'Autentificați-vă cu email și parolă', 'Testați funcția de resetare parolă', and 'Verificați istoricul de scutit după login'.

Figura 5. Prima pagina a aplicatiei.

Înainte de a parcurge aplicația, vom prezenta pe rând funcționalitățile implementate în backend. Primul element analizat este fișierul **init.sql**.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'user_role') THEN
        CREATE TYPE user_role AS ENUM ('USER', 'ANALYST', 'MANAGER');
    END IF;
END $$;

CREATE TABLE IF NOT EXISTS users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    role user_role NOT NULL DEFAULT 'USER',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    locked BOOLEAN NOT NULL DEFAULT false
);

CREATE TABLE IF NOT EXISTS audit_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    action TEXT NOT NULL,
    resource TEXT NOT NULL,
    resource_id TEXT,
    timestamp TIMESTAMPTZ NOT NULL DEFAULT now(),
    ip_address TEXT
);

CREATE TABLE IF NOT EXISTS reset_tokens (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    token TEXT NOT NULL,
    used BOOLEAN NOT NULL DEFAULT false,
    expires_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS idx_audit_logs_user_id ON audit_logs(user_id);
CREATE INDEX IF NOT EXISTS idx_reset_tokens_token ON reset_tokens(token);
```

Fișierul **database/init.sql** definește schema bazei de date PostgreSQL pentru aplicația AuthX. Acesta activează extensia *pgcrypto*, declară tipul *enum user_role* cu valorile USER, ANALYST și MANAGER, după care creează tabelele **users**, **audit_logs** și **reset_tokens**. Tabela **users** este utilizată pentru stocarea datelor conturilor, **audit_logs** reține evenimentele importante din aplicație, iar **reset_tokens** gestionează token-urile necesare pentru resetarea parolei.

Din perspectiva securității, acest fișier configurează doar structura bazei de date, fără a implementa mecanisme avansate de protecție. De exemplu, coloana **password_hash** permite stocarea parolilor, însă nu asigură automat că acestea sunt hash-uite corespunzător. De asemenea, tabela **reset_tokens** poate conține token-uri fără o dată de expirare impusă, iar logurile pot include informații sensibile dacă nu sunt filtrate corespunzător de aplicație. Prin urmare, nivelul real de securitate depinde de modul în care backend-ul utilizează această schemă.

În continuare, vom analiza fișierul **db.js**, care acționează ca un modul auxiliar (helper) pentru executarea interogărilor în baza de date.

```
import pg from "pg";

const { Pool } = pg;

export const pool = new Pool({
  connectionString:
    process.env.DATABASE_URL ||
    "postgres://authx:authx@localhost:5432/authx_vulnerable"
});

export async function query(text, params) {
  return pool.query(text, params);
}

export async function logAudit({ userId = null, action, resource, resourceId = null, ipAddress }) {
  await query(
    `INSERT INTO audit_logs (user_id, action, resource, resource_id, ip_address)
    VALUES ($1, $2, $3, $4, $5)`,
    [userId, action, resource, resourceId, ipAddress]
  );
}
```

Fișierul **backend/src/db.js** configurează conexiunea dintre backend-ul Express și baza de date PostgreSQL. Acesta creează un *Pool* folosind biblioteca *pg*, utilizând variabila de mediu **DATABASE_URL** sau, în lipsa acesteia, o conexiune locală implicită către baza de date *authx_vulnerable*. În același fișier sunt definite funcțiile **query**, care gestionează centralizat execuția interogărilor SQL, și **logAudit**, responsabilă de înregistrarea evenimentelor în tabela **audit_logs**.

Din punct de vedere al securității, implementarea este una de bază, dar utilizează interogări parametrizate atunci când sunt procesate valori, ceea ce reduce riscul de atacuri de tip SQL injection. Cu toate acestea, nu sunt implementate mecanisme suplimentare precum retry controlat pentru conexiuni, criptarea datelor sensibile sau filtrarea informațiilor salvate în audit. În consecință, nivelul de securitate depinde în mare măsură de modul în care celelalte endpoint-uri folosesc aceste funcționalități.

1.2 Prezentarea inițială a backend-ului

În continuare, analizăm fișierul **backend/src/server.js**, care conține logica principală a backend-ului Express pentru aplicația AuthX. În acest fișier sunt configurate middleware-urile necesare, precum CORS, parsarea JSON, gestionarea cookie-urilor și identificarea utilizatorului curent pe baza sesiunii.

De asemenea, aici sunt definite rutele principale ale aplicației: **register**, **login**, **logout**, **forgot-password**, **reset-password**, verificarea utilizatorului curent și afișarea audit logs. Serverul gestionează sesiuni în memorie, setează cookie-ul de autentificare, înregistrează evenimentele importante în audit și interacționează cu baza de date prin funcțiile definite în **db.js**.

În versiunea inițială, implementarea este în mod intenționat nesecurizată. Parolele sunt stocate și comparate în format clar, mesajele de eroare permit identificarea utilizatorilor existenți (*user enumeration*), token-urile de resetare sunt predictibile și pot fi reutilizate, iar cookie-urile nu includ atribute de securitate precum **HttpOnly**, **Secure** sau **SameSite**. În plus, procesul de logout nu invalidează complet sesiunea. Toate aceste aspecte fac ca acest fișier să fie potrivit pentru demonstrarea vulnerabilităților, urmând ca ulterior să fie îmbunătățit din punct de vedere al securității, pas cu pas.

1.2.1. Înregistrare utilizator

Formularul de înregistrare a utilizatorului este afișat în frontend, iar procesarea datelor se realizează în backend prin ruta corespunzătoare. Inițial, se verifică existența adresei de email în baza de date, folosind un query cu clauza **WHERE**. Este important de menționat că aplicația returnează un mesaj diferit atunci când utilizatorul există deja.

Ulterior, noul utilizator este inserat în baza de date, împreună cu parola stocată în clar. Câmpul **locked** este utilizat pentru a limita numărul de încercări de autentificare ale unui utilizator. De asemenea, în cadrul înregistrării este salvat și rolul asociat contului.

```
app.post("/api/auth/register", async (req, res) => {
  const { email, password, role = "USER" } = req.body;

  if (!email || !password) {
    return res.status(400).json({ message: "Email si parola sunt obligatorii." });
  }

  const existing = await query("SELECT id FROM users WHERE email = $1", [email]);
  if (existing.rowCount > 0) {
    await logAudit({
      userId: existing.rows[0].id,
      action: "REGISTER_DUPLICATE",
    });
  }
});
```

```

        resource: "auth",
        resourceId: email,
        ipAddress: req.ip
    });
    return res.status(409).json({ message: "Utilizatorul exista deja." });
}

// V1 vulnerabil: parola este stocata in clar in coloana password_hash.
const result = await query(
    `INSERT INTO users (email, password_hash, role)
    VALUES ($1, $2, $3)
    RETURNING id, email, role, created_at, locked`,
    [email, password, role]
);

await logAudit({
    userId: result.rows[0].id,
    action: "REGISTER",
    resource: "auth",
    resourceId: result.rows[0].id,
    ipAddress: req.ip
});

return res.status(201).json({
    message: "Cont creat. V1 accepta parole slabe intentionat.",
    user: result.rows[0]
});
});

```

1.2.2. Autentificare

Formularul de autentificare este disponibil în frontend, iar procesul de autentificare este gestionat în backend printr-un endpoint dedicat. În cadrul acestuia, se execută o interogare în baza de date folosind o clauză **WHERE** pentru a identifica utilizatorul. În cazul în care acesta nu există, aplicația returnează un mesaj specific.

Se observă că mecanismul **locked** nu este utilizat pentru a limita încercările de autentificare, iar cookie-urile de sesiune nu includ setări de securitate, ceea ce poate expune aplicația la diverse vulnerabilități.

```

app.post("/api/auth/login", async (req, res) => {
    const { email, password } = req.body;

    if (!email || !password) {
        return res.status(400).json({ message: "Email si parola sunt obligatorii." });
    }

    const result = await query("SELECT * FROM users WHERE email = $1", [email]);
    const user = result.rows[0];

```

```

// V1 vulnerabil: raspuns diferit pentru user inexistent.
if (!user) {
  await logAudit({
    action: "LOGIN_UNKNOWN_USER",
    resource: "auth",
    resourceId: email,
    ipAddress: req.ip
  });
  return res.status(404).json({ message: "User inexistent." });
}

// V1 vulnerabil: comparatie directa cu parola in clar, fara hash.
if (user.password_hash !== password) {
  await logAudit({
    userId: user.id,
    action: "LOGIN_FAILED_PASSWORD",
    resource: "auth",
    resourceId: user.id,
    ipAddress: req.ip
  });
  return res.status(401).json({ message: "Parola gresita." });
}

const token = createWeakSession(user);

// V1 vulnerabil: cookie-ul nu are HttpOnly, Secure sau SameSite strict.
res.cookie("authx_session", token, {
  maxAge: 1000 * 60 * 60 * 24 * 30
});

await logAudit({
  userId: user.id,
  action: "LOGIN",
  resource: "auth",
  resourceId: user.id,
  ipAddress: req.ip
});

return res.json({
  message: "Login reusit.",
  token,
  user: {
    id: user.id,
    email: user.email,
    role: user.role,
    locked: user.locked
  }
});
});

```

1.2.3. Deconectare

Funcționalitatea de logout este implementată sub forma unei rute în backend, însă nu este securizată corespunzător, deoarece nu invalidează token-ul sau sesiunea existentă.

```
app.post("/api/auth/logout", async (req, res) => {
  if (req.user) {
    await logAudit({
      userId: req.user.id,
      action: "LOGOUT",
      resource: "auth",
      resourceId: req.user.id,
      ipAddress: req.ip
    });
  }

  // V1 vulnerabil: sterge cookie-ul, dar nu invalideaza token-ul din sessions.
  res.clearCookie("authx_session");
  return res.json({ message: "Logout facut doar in browser; token-ul vechi ramane reutilizabil." });
});
```

1.2.4. Resetare parola

Pentru resetarea parolei, se realizează un *join* între tabelele **reset_tokens** și **users**, în vederea identificării token-ului asociat utilizatorului. După validare, token-ul este invalidat, iar parola este actualizată în baza de date printr-o interogare SQL de tip **UPDATE**.

```
app.post("/api/auth/forgot-password", async (req, res) => {
  const { email } = req.body;

  if (!email) {
    return res.status(400).json({ message: "Email obligatoriu." });
  }

  const result = await query("SELECT id, email FROM users WHERE email = $1", [email]);
  const user = result.rows[0];

  // V1 vulnerabil: raspunsul confirma daca un cont exista.
  if (!user) {
    await logAudit({
      action: "RESET_UNKNOWN_USER",
      resource: "auth",
      resourceId: email,
    });
  }
});
```



```

        ipAddress: req.ip
    });
    return res.status(404).json({ message: "Nu exista cont cu acest email." });
}

const token = createPredictableResetToken(user.email);

await query("INSERT INTO reset_tokens (user_id, token) VALUES ($1, $2)",
[user.id, token]);
await logAudit({
    userId: user.id,
    action: "RESET_TOKEN_CREATED",
    resource: "auth",
    resourceId: user.id,
    ipAddress: req.ip
});

return res.json({
    message: "Token generat. V1 il returneaza direct in raspuns.",
    resetToken: token
});
});
});

```

```

app.post("/api/auth/reset-password", async (req, res) => {
    const { token, newPassword } = req.body;

    if (!token || !newPassword) {
        return res.status(400).json({ message: "Token si parola noua sunt obligatorii." });
    }

    const result = await query(
        `SELECT rt.id, rt.user_id, u.email
        FROM reset_tokens rt
        JOIN users u ON u.id = rt.user_id
        WHERE rt.token = $1
        ORDER BY rt.created_at DESC
        LIMIT 1`,
        [token]
    );

    const reset = result.rows[0];
    if (!reset) {
        return res.status(400).json({ message: "Token invalid." });
    }

    // V1 vulnerabil: token-ul nu expira, nu este one-time si parola ramane in clar.
    await query("UPDATE users SET password_hash = $1 WHERE id = $2", [newPassword,
reset.user_id]);

```

```

await logAudit({
  userId: reset.user_id,
  action: "PASSWORD_RESET",
  resource: "auth",
  resourceId: reset.user_id,
  ipAddress: req.ip
});

return res.json({
  message: `Parola pentru ${reset.email} a fost schimbata. Token-ul poate fi reutilizat in V1.`
});
});

```

1.3 Prezentarea frontend

Frontend-ul este o aplicație React minimalistă, structurată în jurul componentei principale **App** și a două componente secundare: **AuthForms** și **Dashboard**. Componenta **AuthForms** gestionează interfața pentru autentificare, înregistrare, recuperare parolă și resetare parolă, organizate sub formă de taburi. După autentificare, este afișată componenta **Dashboard**, care prezintă informațiile utilizatorului logat și lista de **audit logs**.

Comunicarea cu backend-ul se realizează prin funcția **apiRequest**, care utilizează *fetch* pentru a trimite request-uri către API-ul Express și include automat cookie-urile de sesiune.

```

async function apiRequest(path, options = {}) {
  const token = localStorage.getItem("authx_token");

  const response = await fetch(`${API_URL}${path}`, {
    credentials: "include",
    ...options,
    headers: {
      "Content-Type": "application/json",
      ...(token ? { Authorization: `Bearer ${token}` } : {})
    }
  });

  const data = await response.json();
  if (!response.ok) {
    throw new Error(data.message || "Request esuat.");
  }

  return data;
}

```

Capitolul 2. Securizarea aplicației

Vom începe prin a crea trei utilizatori, fiecare având roluri diferite: **admin**, **analyst** și **user**, pentru a putea testa funcționalitățile aplicației și diferențele de acces între roluri, vizibil în Video 1 și Video 2

2.1. Password policy slab

Folosind aplicația Insomnia [3], vom demonstra că este posibilă crearea unui cont utilizând o parolă slabă. Pentru aceasta, vom orchestra următorul request:

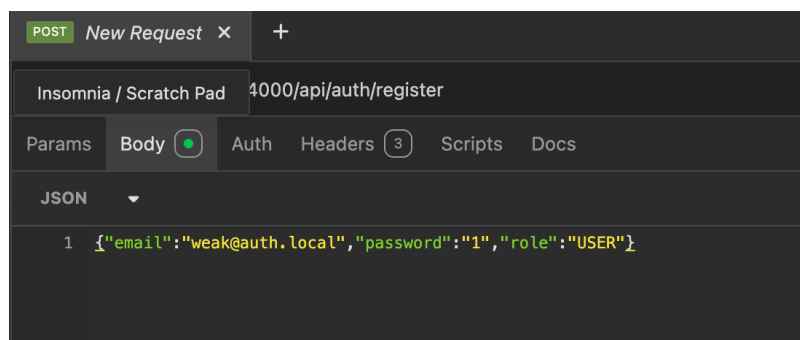


Figura 6. Request-ul cu body.

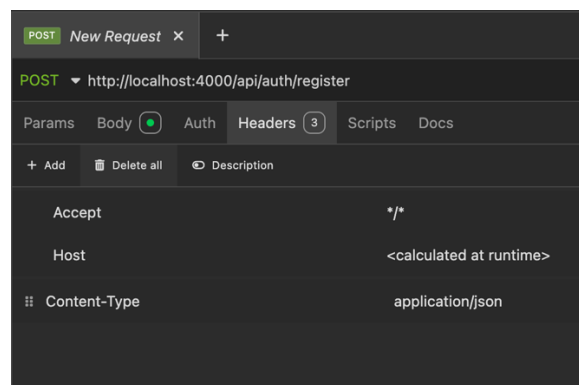


Figura 7. Request-ul cu headers.

```

1 {
2   "message": "Cont creat. V1 accepta parole
   slabe intentionat.",
3   "user": {
4     "id": "1876efea-d347-4c17-b08f-
      10313c693c79",
5     "email": "weak@auth.local",
6     "role": "USER",
7     "created_at": "2026-04-26T05:45:35.165Z",
8     "locked": false
9   }
10 }

```

Figura 8. Răspunsul.

Iar după autentificare, observăm că procesul funcționează corespunzător, iar utilizatorul poate accesa aplicația.

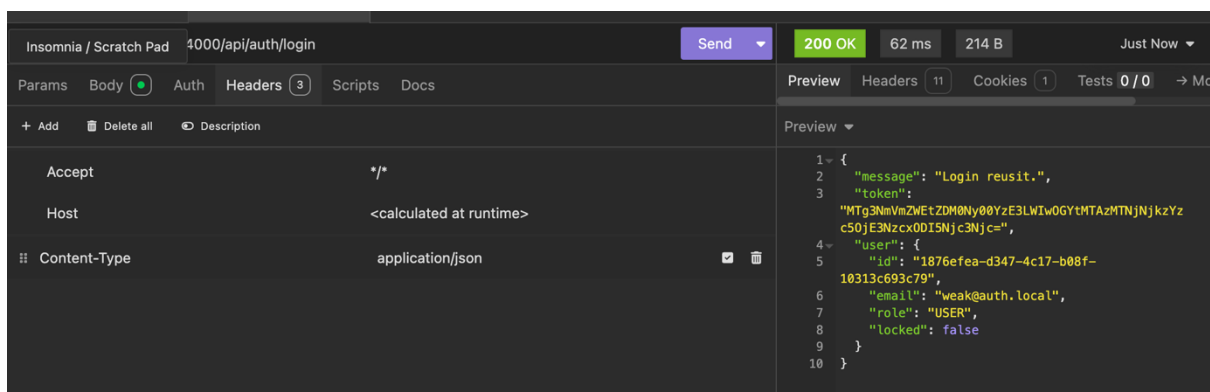


Figura 9. Login cu contul slab.

Acest comportament reprezintă o vulnerabilitate, deoarece un atacator ar putea compromite conturile utilizatorilor prin parole slabe. Pentru a demonstra acest risc, se poate utiliza Hydra [4], un tool de tip *brute-force* care încercă automat parole dintr-un fișier predefinit.

În acest caz, a fost utilizat un dicționar de parole descărcat din SecLists, disponibil aici: <https://github.com/danielmiessler/SecLists/blob/master/Passwords/Default-Credentials/default-passwords.txt>

Pentru testare, se rulează următoarea comandă:

```
hydra -V -l admin@authx.local -P ./default-passwords.txt 127.0.0.1 -s 4000 http-post-form  
'/api/auth/login:{"email":"^USER^","password":"^PASS^"}:S=Login reusit:H=Content-  
Type\: application/json'
```

Se poate observa în Video 4 rezultatul: parola slabă „adm” este identificată foarte rapid, ceea ce evidențiază vulnerabilitatea sistemului.

Pentru a remedia această problemă, este necesară implementarea unor mecanisme de validare a parolilor. O soluție eficientă este utilizarea librăriei npm **zxcvbn** [5] care permite evaluarea complexității unei parole și detectarea celor slabe sau ușor de ghicit.

```
import zxcvbn from 'zxcvbn';  
const PASSWORD_POLICY_MESSAGE = 'Parola nu respecta politica de securitate.';  
const MIN_PASSWORD_LENGTH = 10;  
const MIN_PASSWORD_SCORE = 8;  
function validatePasswordPolicy(password, userInputs = []) {  
  if (typeof password !== 'string' || password.length < MIN_PASSWORD_LENGTH) {  
    return false;  
  }  
  
  const passwordStrength = zxcvbn(password, userInputs);  
  return passwordStrength.score >= MIN_PASSWORD_SCORE;  
}  
app.post('/api/auth/register', async (req, res) => {  
  const { email, password, role = 'USER' } = req.body;  
  
  if (!email || !password) {  
    return res  
      .status(400)  
      .json({ message: 'Email si parola sunt obligatorii.' });  
  }  
  
  if (!validatePasswordPolicy(password, [email])) {  
    return res.status(400).json({ message: PASSWORD_POLICY_MESSAGE });  
  }  
}
```

Acum vom testa același request în Insomnia. După cum se poate observa în Video 5, aplicația nu mai permite crearea contului cu o parolă slabă.

2.2. Stocare nesigură a parolelor

Problema stocării parolelor în clar este critică, deoarece în cazul unui *database leak*, atacatorii ar obține acces direct la parolele utilizatorilor. Ulterior, aceste parole ar putea fi încercate și pe alte platforme, ceea ce poate duce la compromiterea conturilor personale ale utilizatorilor.

Pentru a simula acest scenariu, vom mima accesul la baza de date prin executarea unui query SQL.

```
dobriceanioandorian-351 breakthellogin-ioandobricean git:(fixed) x docker exec -it authx-postgres psql -U authx -d authx_vulnerable \
-c "SELECT email, password_hash FROM users ORDER BY created_at DESC LIMIT 5;"
```

email	password_hash
weak@authx.local	1
user@authx.local	use
analyst@authx.local	ana
admin@authx.local	adm
administrator@authx.local	adm

```
(5 rows)
```

```
dobriceanioandorian-351 breakthellogin-ioandobricean git:(fixed) x
```

Figura 6. Acces la baza de date.

Pentru a remedia această problemă, vom aplica o practică standard de securitate în ExpressJS: stocarea parolelor sub formă de hash, folosind algoritmul **scrypt** [6].

scrypt este o funcție de derivare a cheilor (*key derivation function*) concepută pentru a fi rezistentă la atacuri de tip *brute-force*. Aceasta transformă parola într-un hash printr-un proces costisitor din punct de vedere computațional și de memorie, ceea ce face mult mai dificilă încercarea rapidă a unui număr mare de parole de către un atacator.

În plus, vom utiliza și un **salt**, adică o valoare aleatoare unică generată pentru fiecare parolă înainte de hashuire. Acest salt este concatenat cu parola și apoi procesat de scrypt, rezultând un hash diferit chiar și pentru parole identice. Astfel, se previne utilizarea tabelor precompute (*rainbow tables*) și se crește semnificativ nivelul de securitate al parolelor stocate.

```
import { randomBytes, scrypt, timingSafeEqual } from 'node:crypto';
async function hashPassword(password) {
  const salt = randomBytes(16).toString('hex');
  const derivedKey = await scryptAsync(password, salt, PASSWORD_KEY_LENGTH);
  return `${PASSWORD_HASH_PREFIX}${salt}${derivedKey.toString('hex')}`;
}

app.post('/api/auth/register', async (req, res) => {
  const { email, password, role = 'USER' } = req.body;
```

```

if (!email || !password) {
  return res
    .status(400)
    .json({ message: 'Email si parola sunt obligatorii.' });
}

if (!validatePasswordPolicy(password, [email])) {
  return res.status(400).json({ message: PASSWORD_POLICY_MESSAGE });
}

const existing = await query('SELECT id FROM users WHERE email = $1', [
  email,
]);
if (existing.rowCount > 0) {
  await logAudit({
    userId: existing.rows[0].id,
    action: 'REGISTER_DUPLICATE',
    resource: 'auth',
    resourceId: email,
    ipAddress: req.ip,
  });
  return res.status(409).json({ message: 'Utilizatorul exista deja.' });
}

const passwordHash = await hashPassword(password);
const result = await query(
  `INSERT INTO users (email, password_hash, role)
  VALUES ($1, $2, $3)
  RETURNING id, email, role, created_at, locked`,
  [email, passwordHash, role],
);

await logAudit({
  userId: result.rows[0].id,
  action: 'REGISTER',
  resource: 'auth',
  resourceId: result.rows[0].id,
  ipAddress: req.ip,
});

return res.status(201).json({
  message: 'Cont creat.',
  user: result.rows[0],
});
});

```

Acum, folosind aplicația web, vom crea un nou cont, după care vom verifica în baza de date ce informații sunt stocate. Rezultatul poate fi observat în Video 6.

```
Last login: Sun Apr 26 09:39:24 on ttys001
kdobricean@oandorian-351 ~$ cd breakthelogin-iaandobricean
kdobricean@oandorian-351 breakthelogin-iaandobricean$ git:(fixed) ✗ docker exec -it authx-postgres psql -U authx -d authx_vulnerable \
-c "SELECT email, password_hash FROM users ORDER BY created_at DESC LIMIT 5;"

```

email	password_hash
usersecurizat@authx.local	scrypt\$5f8972595168f013fc5b56d3b537a336\$7abb8b7aa003054c18a832709c3f7449905a850f9d97b332cc2a76327601531e06f3a9c9e7f0dceca0fb6122db36b01a4b4ffc9172c4a1ec89a22c5cb20bd752
weak@authx.local	1
user@authx.local	use
analyst@authx.local	ana
admin@authx.local	adm

```
(5 rows)
kdobricean@oandorian-351 breakthelogin-iaandobricean$ git:(fixed) ✗
```

Figura 7. Parola securizată în baza de date.

2.3 Brute Force

Pentru a evidenția vulnerabilitatea la atacuri de tip *brute-force*, vom utiliza din nou Hydra. De această dată, vom folosi utilizatorul **analyst** (prezentat în figura anterioară), care are parola „ana”. Deși aceasta nu este o parolă sigură, contul a fost creat înainte de implementarea mecanismelor de securitate pentru parole, ceea ce îl face potrivit pentru demonstrație.

Scopul este de a arăta că, utilizând un fișier de parole suficient de mare, Hydra poate identifica parola corectă și obține acces în cont. Acest tip de atac este ilustrat în Video 7.

Pentru implementarea acestui mecanism, vom adăuga mai întâi câteva câmpuri suplimentare în baza de date, care nu existau în versiunea inițială a aplicației. Aceste câmpuri vor permite monitorizarea încercărilor eșuate de autentificare și aplicarea blocării temporare a contului atunci când este depășit numărul maxim de încercări permise.

```
ALTER TABLE users
ADD COLUMN IF NOT EXISTS failed_login_attempts INTEGER NOT NULL DEFAULT 0;

ALTER TABLE users
ADD COLUMN IF NOT EXISTS locked_until TIMESTAMPTZ;
```

Aceste coloane ne ajută să determinăm până când un utilizator rămâne blocat și în ce moment trebuie aplicată blocarea contului.

În **server.js**, vom verifica dacă numărul de încercări eșuate depășește valoarea **MAX_FAILED_ATTEMPTS**. Dacă limita este depășită, contul va fi blocat temporar și se va salva momentul blocării. Ulterior, dacă intervalul de blocare a expirat, contul va fi deblocat automat, iar utilizatorul va putea încerca din nou autentificarea.

```
function isAccountLocked(user) {
  return user.locked && user.locked_until && new Date(user.locked_until) > new Date();
}

if (isAccountLocked(user)) {
  await logAudit({
    userId: user.id,
    action: 'LOGIN_BLOCKED_LOCKED',
    resource: 'auth',
```



```

        resourceId: user.id,
        ipAddress: req.ip,
    });
    return res.status(423).json({
        message: 'Cont blocat temporar. Incearca mai tarziu.',
    });

    if (user.locked && user.locked_until && new Date(user.locked_until) <= new
Date()) {
        await query(
            `UPDATE users
             SET locked = false, failed_login_attempts = 0, locked_until = NULL
             WHERE id = $1`,
            [user.id],
        );
        user.locked = false;
        user.failed_login_attempts = 0;
        user.locked_until = null;
    }

app.post('/api/auth/login', async (req, res) => {
    const { email, password } = req.body;

    if (!email || !password) {
        return res
            .status(400)
            .json({ message: 'Email si parola sunt obligatorii.' });
    }

    const result = await query('SELECT * FROM users WHERE email = $1', [email]);
    const user = result.rows[0];

    // V1 vulnerabil: raspuns diferit pentru user inexistent.
    if (!user) {
        await logAudit({
            action: 'LOGIN_UNKNOWN_USER',
            resource: 'auth',
            resourceId: email,
            ipAddress: req.ip,
        });
        return res.status(404).json({ message: 'User inexistent.' });
    }

    if (isAccountLocked(user)) {
        await logAudit({
            userId: user.id,
            action: 'LOGIN_BLOCKED_LOCKED',
            resource: 'auth',
            resourceId: user.id,
            ipAddress: req.ip,
        });
    }

```

```

});
return res.status(423).json({
  message: 'Cont blocat temporar. Incearca mai tarziu.',
});
}

if (user.locked && user.locked_until && new Date(user.locked_until) <= new
Date()) {
  await query(
    `UPDATE users
      SET locked = false, failed_login_attempts = 0, locked_until = NULL
      WHERE id = $1`,
    [user.id],
  );
  user.locked = false;
  user.failed_login_attempts = 0;
  user.locked_until = null;
}

const passwordCheck = await verifyPassword(password, user.password_hash);
if (!passwordCheck.valid) {
  const failedAttempts = Number(user.failed_login_attempts || 0) + 1;
  const shouldLock = failedAttempts >= MAX_FAILED_LOGIN_ATTEMPTS;

  await query(
    `UPDATE users
      SET failed_login_attempts = $1,
        locked = $2,
        locked_until = CASE
          WHEN $2 THEN now() + ($3 || ' minutes')::interval
          ELSE locked_until
        END
      WHERE id = $4`,
    [failedAttempts, shouldLock, ACCOUNT_LOCK_MINUTES, user.id],
  );

  await logAudit({
    userId: user.id,
    action: shouldLock ? 'LOGIN_ACCOUNT_LOCKED' : 'LOGIN_FAILED_PASSWORD',
    resource: 'auth',
    resourceId: user.id,
    ipAddress: req.ip,
  });

  if (shouldLock) {
    return res.status(423).json({
      message: 'Cont blocat temporar. Incearca mai tarziu.',
    });
  }

  return res.status(401).json({ message: 'Parola gresita.' });
}

```

```

}

if (passwordCheck.needsRehash) {
  const upgradedHash = await hashPassword(password);
  await query('UPDATE users SET password_hash = $1 WHERE id = $2', [
    upgradedHash,
    user.id,
  ]);
}

await query(
  `UPDATE users
  SET failed_login_attempts = 0, locked = false, locked_until = NULL
  WHERE id = $1`,
  [user.id],
);

const token = createWeakSession(user);

// V1 vulnerabil: cookie-ul nu are HttpOnly, Secure sau SameSite strict.
res.cookie('authx_session', token, {
  maxAge: 1000 * 60 * 60 * 24 * 30,
});

await logAudit({
  userId: user.id,
  action: 'LOGIN',
  resource: 'auth',
  resourceId: user.id,
  ipAddress: req.ip,
});

return res.json({
  message: 'Login reusit.',
  token,
  user: {
    id: user.id,
    email: user.email,
    role: user.role,
    locked: user.locked,
  },
});
});

```

Acum vom retesta aceeași comandă pentru a observa comportamentul aplicației. Se poate vedea că Hydra încearcă mai multe parole, însă nu obține niciun rezultat valid, în Video 8. Deși Hydra nu indică explicit faptul că respectivul cont a fost blocat, verificarea în Insomnia confirmă că mecanismul de blocare funcționează, iar contul este într-adevăr blocat.

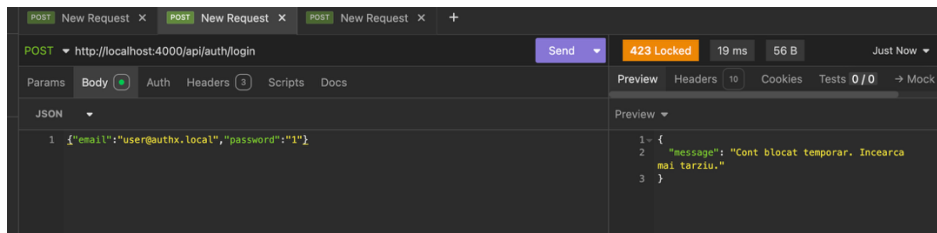


Figura 7. Cont blocat.

2.4 User Enumeration

Un atacator poate identifica ce adrese de email au deja cont în aplicație. Pentru a demonstra această vulnerabilitate, vom utiliza **ffuf** [7].

ffuf funcționează într-un mod asemănător cu Hydra, însă scopul său este de a trimite request-uri multiple și de a analiza răspunsurile primite din partea backend-ului. În acest caz, vom crea un fișier **emails.txt**, care va conține adresele de email testate de ffuf.

```
admin@authx.local
analyst@authx.local
manager@authx.local
user@authx.local
nu-exista@authx.local
test@authx.local
```



Figura 8. Ffuf.

Se observă vulnerabilitatea: adresele de email existente returnează codul **401**, iar cele inexistente returnează **404**, permițând astfel identificarea conturilor valide.

Pentru a remedia această problemă, vom uniformiza răspunsurile serverului, astfel încât să fie același în ambele cazuri, eliminând posibilitatea de *user enumeration*.

```
if (!user) {
  await logAudit({
    action: 'LOGIN_UNKNOWN_USER',
    resource: 'auth',
    resourceId: email,
    ipAddress: req.ip,
  });
  return res.status(401).json({ message: INVALID_CREDENTIALS_MESSAGE });
}
return res.status(401).json({ message: INVALID_CREDENTIALS_MESSAGE });
```

Iar utilizând din nou ffuf, vom obține următorul rezultat, în care nu mai putem diferenția între email-urile existente și cele inexistente, confirmând că vulnerabilitatea a fost remediată.

```
kdobriceanioandorian-351 breakthelgin-ioandobricean git:(Fixed) x ffuf \
-w emails.txt:EMAIL \
-u http://localhost:4000/api/auth/login \
-X POST \
-H "Content-Type: application/json" \
-d '{"email":"EMAIL","password":"parola-gresita"}' \
-mc all \
-mr "Parola gresita."

v2.1.0-dev

:: Method      : POST
:: URL         : http://localhost:4000/api/auth/login
:: Wordlist     : EMAIL: /Users/bucur-sabauaria/breakthelgin-ioandobricean/emails.txt
:: Header      : Content-Type: application/json
:: Data        : {"email":"EMAIL","password":"parola-gresita"}
:: Follow redirects : false
:: Calibration  : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: all
:: Matcher     : Regexp: Parola gresita.

manager@authx.local [Status: 401, Size: 35, Words: 2, Lines: 1, Duration: 69ms]
nu-exista@authx.local [Status: 401, Size: 35, Words: 2, Lines: 1, Duration: 70ms]
admin@authx.local [Status: 401, Size: 35, Words: 2, Lines: 1, Duration: 70ms]
test@authx.local [Status: 401, Size: 35, Words: 2, Lines: 1, Duration: 70ms]
user@authx.local [Status: 401, Size: 35, Words: 2, Lines: 1, Duration: 71ms]
analyst@authx.local [Status: 401, Size: 35, Words: 2, Lines: 1, Duration: 100ms]
:: Progress: [6/6] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Errors: 0 ::
kdobriceanioandorian-351 breakthelgin-ioandobricean git:(Fixed) x
```

Figura 8. Ffuf cu toate raspunsul 401.

2.5 Gestionarea nesigură a sesiunilor

Pentru a demonstra vulnerabilitatea asociată cookie-urilor fără atributul **HttpOnly**, vom accesa aplicația web și vom rula în consola browserului următoarea comandă:

```
> document.cookie
< 'I18N_LANGUAGE=en; toolbar_expanded=true; __stripe_mid=3d208fba-ac5e-4909-b566-c1b0afc088a2c34d63; sidebar_expanded=true; authx_session=0Dc2MGZiMDItOGI3OC00YjdkLWIyYTEtNGZmOTkxNTNiZGZk0jE3NzcwOTQzMDI2NTA%3D'
```

Figura 9. Cookie fără HttpOnly.

Pentru a securiza această problemă, vom seta atributele corespunzătoare pentru cookie-uri, precum **HttpOnly**, **Secure** și **SameSite**, astfel încât acestea să nu mai poată fi accesate din JavaScript și să fie protejate împotriva atacurilor de tip XSS și CSRF. În plus, vom reduce durata de viață a token-ului, pentru a limita fereastra de timp în care acesta poate fi utilizat în mod abuziv.

```
res.cookie('authx_session', token, {
  httpOnly: true,
  secure: process.env.NODE_ENV === 'production',
  sameSite: 'strict',
  maxAge: 1000 * 60 * 30;
});
```

Acum, aceeași comandă va returna un rezultat gol (sau nu va mai afișa cookie-ul de sesiune), demonstrând că acesta nu mai este accesibil din JavaScript datorită atributului **HttpOnly**.

```
experience: https://reactjs.org/link/react-devtools
> document.cookie
< ''

Navigated to http://localhost:5173/
react-dom client.js?v=12b0fb66:2
Download the React DevTools for a better development
```

Figura 9. Cookie securizat.

2.6 Resetare parolă nesigură

```
function invalidateUserSessions(userId) {
  for (const [token, session] of sessions.entries()) {
    if (session.userId === userId) {
      sessions.delete(token);
    }
  }
}
```

```
}  
}
```

```
invalidateUserSessions(user.id);  
const token = createSession(user);  
  
res.cookie('authx_session', token, {  
  ...SESSION_COOKIE_OPTIONS,  
  maxAge: SESSION_MAX_AGE_MS,  
});
```

2.6 Resetare parolă nesigură

În primul rând prima vulnerabilitate în resetarea parolelor este faptul ca endpointul retrimite reset tokenul în răspuns

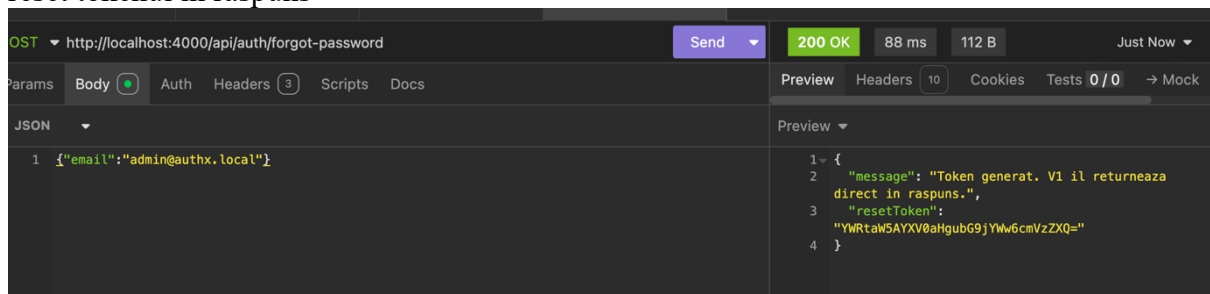


Figura 10. Utilizare forgot-password nesecurizat.

O altă vulnerabilitate este faptul că reset tokenul este unul previzibil, dacă rulăm această comandă observăm ca este identic cu cel din figura 10, astfel un atactor ar putea reseta parola pentru orice email găsit prin User Enumeration

```
node -e "console.log(Buffer.from('admin@authx.local:reset').toString('base64'))"
```

Acum vom reseta parola folosind tokenul primit

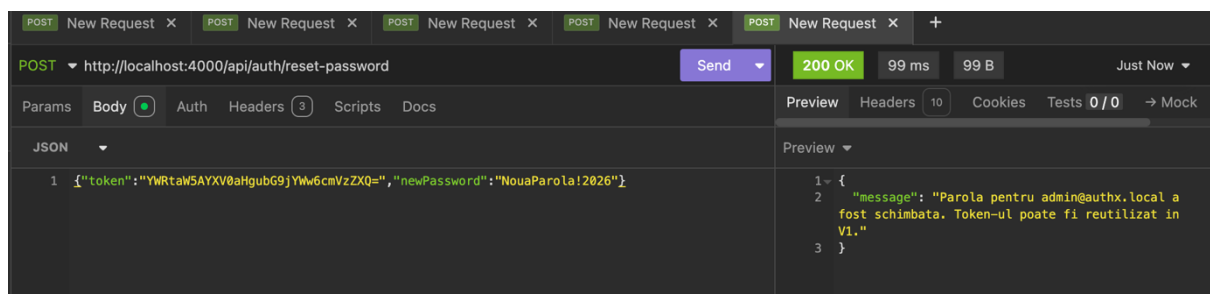


Figura 11. Resetare Parolă

Însă observăm că și după trecerea timpului același token încă funcționează

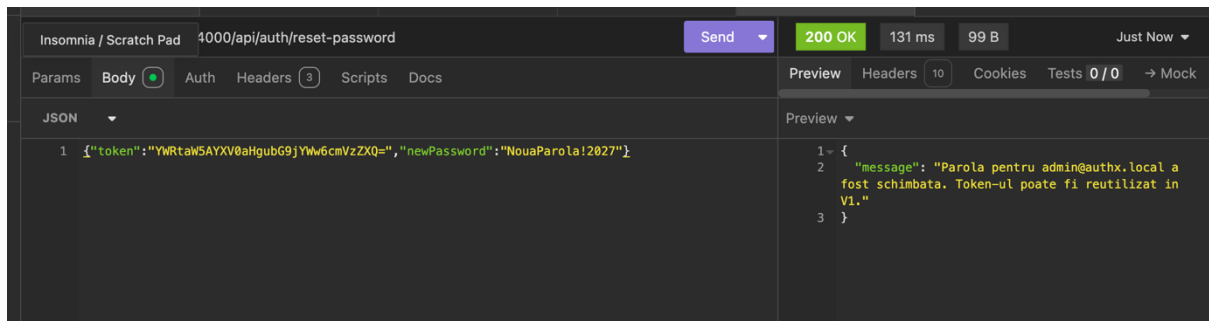


Figura 12. Resetare Parolă peste 5 min

Pentru securizarea procesului de resetare a parolei, în tabela **reset_tokens** trebuie adăugat câmpul **expires_at TIMESTAMPTZ NOT NULL**. Acesta stabilește momentul până la care token-ul este valid. Astfel, un token de resetare nu mai poate fi folosit pe termen nelimitat, ci doar într-un interval scurt, definit prin variabila **RESET_TOKEN_TTL_MINUTES**.

Token-ul este generat folosind funcția **createResetToken()**, care produce o valoare aleatoare de 32 de bytes, convertită în format hexazecimal. Acest lucru face ca token-ul să fie greu de ghicit sau anticipat. Înainte de salvarea în baza de date, token-ul este trecut prin funcția **hashResetToken()**, care aplică algoritmul **SHA-256**. Astfel, în baza de date nu se salvează token-ul real, ci doar hash-ul acestuia. Dacă baza de date este compromisă, atacatorul nu poate folosi direct token-urile pentru resetarea parolelor.

În ruta **forgot-password**, aplicația verifică mai întâi dacă email-ul există. Totuși, pentru a evita vulnerabilitatea de tip **user enumeration**, răspunsul trimis către client este același atât pentru email-urile existente, cât și pentru cele inexistente. Dacă utilizatorul există, se generează token-ul, se calculează hash-ul acestuia și se salvează în tabela **reset_tokens** împreună cu data de expirare. În mediul de producție, token-ul nu este returnat în răspuns, ci ar trebui trimis utilizatorului prin email.

```
function createResetToken() {
  return randomBytes(32).toString('hex');
}

function hashResetToken(token) {
  return createHash('sha256').update(token).digest('hex');
}

app.post('/api/auth/forgot-password', async (req, res) => {
  const { email } = req.body;

  if (!email) {
    return res.status(400).json({ message: 'Email obligatoriu.' });
  }

  const result = await query('SELECT id, email FROM users WHERE email = $1', [
    email,
  ]);
  const user = result.rows[0];
```



```

if (!user) {
  await logAudit({
    action: 'RESET_UNKNOWN_USER',
    resource: 'auth',
    resourceId: email,
    ipAddress: req.ip,
  });
  return res.json({
    message:
      'Daca exista un cont pentru acest email, instructiunile de resetare au fost
generate.',
  });
}

const token = createResetToken();
const tokenHash = hashResetToken(token);

await query(
  `INSERT INTO reset_tokens (user_id, token, expires_at)
  VALUES ($1, $2, now() + ($3 || ' minutes')::interval)`,
  [user.id, tokenHash, RESET_TOKEN_TTL_MINUTES],
);
await logAudit({
  userId: user.id,
  action: 'RESET_TOKEN_CREATED',
  resource: 'auth',
  resourceId: user.id,
  ipAddress: req.ip,
});

return res.json({
  message:
    'Daca exista un cont pentru acest email, instructiunile de resetare au fost
generate.',
  ...(process.env.NODE_ENV === 'production' ? {} : { resetToken: token }),
});
});

```

Pentru ruta **reset-password**, token-ul primit de la utilizator este mai întâi hash-uit cu aceeași funcție **hashResetToken()**, apoi este căutat în baza de date. Interogarea verifică trei condiții importante: token-ul trebuie să existe, să nu fi fost folosit anterior și să nu fie expirat. Dacă aceste condiții nu sunt îndeplinite, aplicația returnează mesajul „**Token invalid sau expirat.**”

Dacă token-ul este valid, parola nouă este verificată conform politicii de securitate, apoi este hash-uită și actualizată în tabela **users**. În aceeași tranzacție SQL, token-ul este marcat ca folosit prin setarea câmpului **used = true**. Folosirea unei tranzacții este importantă deoarece garantează că actualizarea parolei și invalidarea token-ului se realizează împreună. Dacă apare o eroare, modificările sunt anulate prin **ROLLBACK**.

```
app.post('/api/auth/reset-password', async (req, res) => {
  const { token, newPassword } = req.body;

  if (!token || !newPassword) {
    return res
      .status(400)
      .json({ message: 'Token si parola noua sunt obligatorii.' });
  }

  if (!validatePasswordPolicy(newPassword)) {
    return res.status(400).json({ message: PASSWORD_POLICY_MESSAGE });
  }

  const tokenHash = hashResetToken(token);
  const result = await query(
    `SELECT rt.id, rt.user_id, u.email
    FROM reset_tokens rt
    JOIN users u ON u.id = rt.user_id
    WHERE rt.token = $1
    AND rt.used = false
    AND rt.expires_at > now()
    ORDER BY rt.created_at DESC
    LIMIT 1`,
    [tokenHash],
  );

  const reset = result.rows[0];
  if (!reset) {
    return res.status(400).json({ message: 'Token invalid sau expirat.' });
  }

  const passwordHash = await hashPassword(newPassword);
  const client = await pool.connect();
  try {
    await client.query('BEGIN');
    await client.query('UPDATE users SET password_hash = $1 WHERE id = $2', [
      passwordHash,
      reset.user_id,
    ]);
    await client.query('UPDATE reset_tokens SET used = true WHERE id = $1',
[reset.id]);
    await client.query('COMMIT');
  } catch (error) {
    await client.query('ROLLBACK');
    throw error;
  } finally {
    client.release();
  }
}
```

```

invalidateUserSessions(reset.user_id);
await logAudit({
  userId: reset.user_id,
  action: 'PASSWORD_RESET',
  resource: 'auth',
  resourceId: reset.user_id,
  ipAddress: req.ip,
});

return res.json({
  message: `Parola pentru ${reset.email} a fost schimbata.`,
});
});

```

După resetarea parolei, sesiunile utilizatorului sunt invalidate prin **invalidateUserSessions(reset.user_id)**, astfel încât eventualele sesiuni vechi să nu mai rămână active. În final, acțiunea este salvată în audit logs, iar aplicația confirmă schimbarea parolei.

Acum observăm ca tokenul nu mai e predictibil:

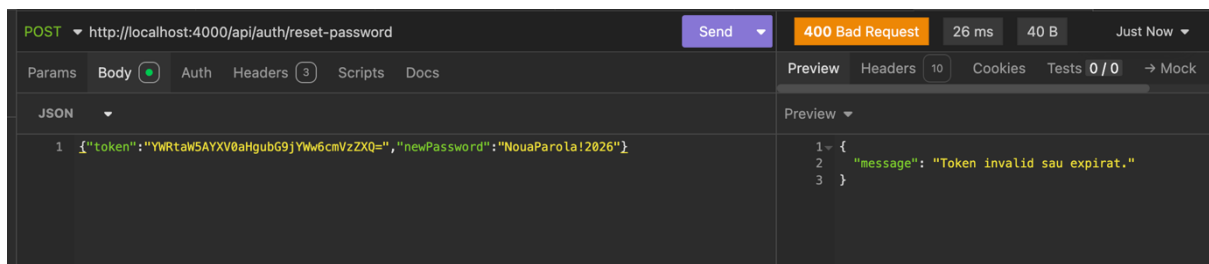


Figura 12. Token invalid

Acum vom demonstra că token-ul nu mai poate fi reutilizat. În primul pas, vom reseta parola cu succes folosind token-ul generat. Deoarece rulăm aplicația în mediul local (non-production), token-ul este încă returnat în răspuns, ceea ce ne permite să îl folosim direct pentru testare.

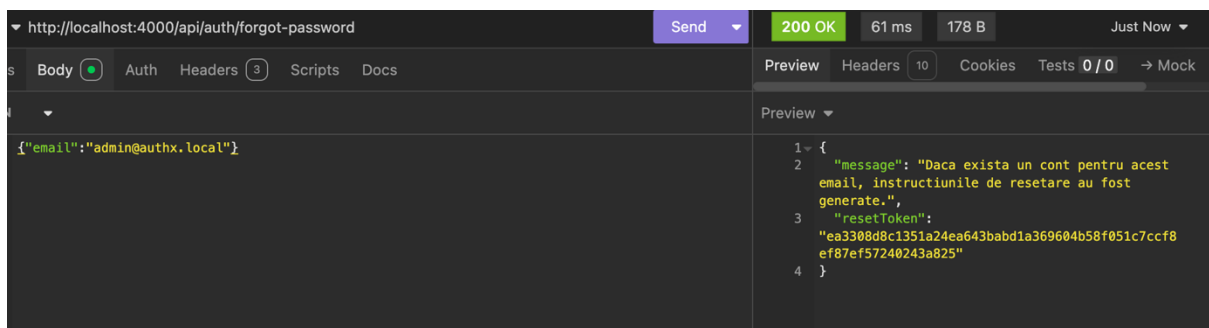


Figura 12. Resetare parolă din nou

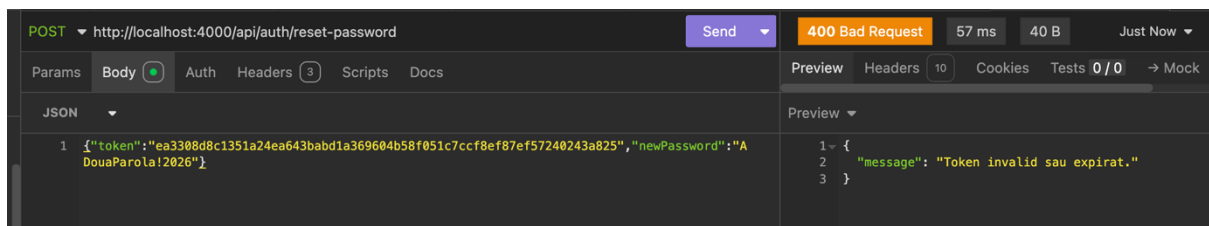


Figura 13. Resetare parolă cu succes

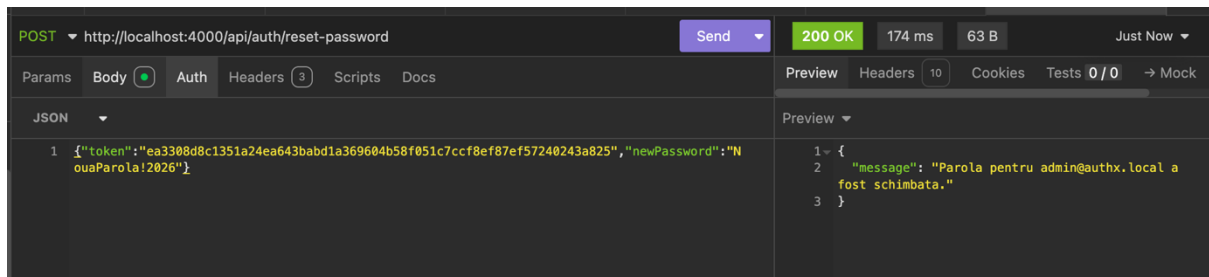


Figura 14. Același token este invalidat

Capitolul 3. Concluzie

În cadrul acestei lucrări a fost analizată și îmbunătățită securitatea unei aplicații web de tip autentificare, construită pe un stack modern: **React – Express – PostgreSQL**. Pornind de la o implementare inițială intenționat vulnerabilă, au fost identificate și demonstrate mai multe riscuri reale de securitate, folosind tool-uri specifice precum **Insomnia**, **Hydra** și **ffuf**.

Printre principalele vulnerabilități analizate s-au numărat: utilizarea parolelor slabe, stocarea parolelor în clar, lipsa mecanismelor de protecție împotriva atacurilor de tip *brute-force*, posibilitatea de *user enumeration*, cookie-uri nesecurizate și token-uri de resetare reutilizabile sau fără expirare. Fiecare dintre aceste probleme a fost demonstrată practic, evidențiind impactul potențial asupra utilizatorilor și aplicației.

Ulterior, au fost implementate soluții de securizare conform bunelor practici din industrie: validarea complexității parolelor folosind **zxcvbn**, hash-uirea parolelor cu **scrypt** și utilizarea unui **salt**, introducerea mecanismului de **rate limiting**, uniformizarea răspunsurilor pentru prevenirea *user enumeration*, securizarea cookie-urilor prin attributele **HttpOnly**, **Secure** și **SameSite**, precum și implementarea unui sistem robust de resetare a parolei bazat pe token-uri unice, hash-uite, cu expirare și invalidare după utilizare.

Rezultatele obținute demonstrează importanța abordării securității încă din fazele incipiente ale dezvoltării unei aplicații. O implementare aparent funcțională poate ascunde vulnerabilități critice, care pot fi exploatare cu ușurință în lipsa unor măsuri adecvate. Prin aplicarea treptată a mecanismelor de protecție, aplicația a evoluat de la un sistem vulnerabil la unul semnificativ mai sigur și mai robust.

În concluzie, securitatea aplicațiilor web nu reprezintă un pas opțional, ci o componentă esențială a dezvoltării software, care trebuie tratată cu prioritate și actualizată constant în funcție de noile tipuri de atacuri și vulnerabilități.

Capitolul 4. Bibliografie

- [1] “Express - Node.js web application framework.” Accessed: Apr. 25, 2026. [Online]. Available: <https://expressjs.com/>
- [2] “React.” Accessed: Apr. 07, 2026. [Online]. Available: <https://react.dev/>
- [3] “The Collaborative API Development Platform.” Accessed: Apr. 26, 2026. [Online]. Available: <https://insomnia.rest/>
- [4] “Hydra,” TryHackMe. Accessed: Apr. 26, 2026. [Online]. Available: <https://tryhackme.com/room/hydra>
- [5] “@types/zxcvbn,” npm. Accessed: Apr. 26, 2026. [Online]. Available: <https://www.npmjs.com/package/@types/zxcvbn>
- [6] “scrypt,” npm. Accessed: Apr. 26, 2026. [Online]. Available: <https://www.npmjs.com/package/scrypt>
- [7] “ffuf,” Homebrew Formulae. Accessed: Apr. 26, 2026. [Online]. Available: <https://formulae.brew.sh/formula/ffuf>